

מודול 4 — סקריפטינג Bash ו-Python — תרגילים ופתרונות

חומר לימוד

2 יחידות

תוכן העניינים

1.  תרגילים — מודול 4: סקריפטינג (Python ו-Bash)

2.  פתרונות מלאים — מודול 4: סקריפטינג

תרגילים — מודול 4: סקריפטינג (Python ו-Bash)

כל הקוד רץ במעבדה שלך. סריקות — רק על 127.0.0.1 או מכונות המעבדה. פתרונות ב- `solutions.md`.

מקרא רמות: קל • בינוני • מתקדם • אתגר

חלק א' — Bash (●●)

4.1 — כתוב סקריפט `hello.sh` עם Shebang שמדפיס "Hello Hacker", הפוך אותו לניתן-הרצה, והרץ. ●

4.2 — כתוב סקריפט שמקבל שם באמצעות `read` ומדפיס `Welcome, <name>`. ●

4.3 — כתוב סקריפט שמקבל מספר פורט כארגומנט (\$1) ומדפיס אם הוא 80 (HTTP), 443 (HTTPS), או Unknown. ●

4.4 — כתוב לולאה ב-Bash שמדפיסה `Pinging 10.0.0.X` עבור X מ-1 עד 5. ●

חלק ב' — Python בסיסי (●●)

4.5 — כתוב סקריפט Python שמגדיר משתנה `target` ומדפיס אותו עם `f-string`. ●

4.6 — נתון `banner = "Apache/2.4.49 (Debian)"`. חלץ והדפס רק את מספר הגרסה (2.4.49). ●

4.7 — צור מילון `services` הממפה 21,22,80,443 לשמות השירותים, ובלולאה הדפס `Port X -> Service`. ●

חלק ג' — לוגיקה ולולאות (●)

4.8 — כתוב פונקציה `check_port(port)` שמחזירה את שם השירות אם הפורט מוכר (מתוך מילון), אחרת "Unknown". ●

4.9 — כתוב לולאה שעוברת על פורטים 20–25 ומדפיסה לכל אחד את שם השירות (בעזרת הפונקציה מ-4.8). ●

חלק ד' — הכלים האמיתיים (מתקדם) 🟡

4.10 — הרץ את **Ping Sweep** (מהחומר) על תת-הרשת של המעבדה שלך. כמה מכונות חיות נמצאו?

4.11 — הרץ את **Port Scanner** (מהחומר) על `127.0.0.1`. אילו פורטים פתוחים? (רמז: הפעל שירות עם `python3 -m http.server 8000` כדי לראות פורט נפתח).

4.12 — קרא את קוד ה-Port Scanner וענה: מה מחזירה `connect_ex` בפורט פתוח? מה תפקיד `settimeout`? מדוע צריך את `try/except KeyboardInterrupt`?

אתגר מסכם — “שדרוג הסורק” 🔴

שדרג את ה-Port Scanner כך שיכלול:

- שם שירות** — לצד כל פורט פתוח, הדפס את שם השירות ממילון `services`.
- טווח מהמשתמש** — קבל את פורט ההתחלה והסיום מהמשתמש (`input`).
- סיכום** — בסוף: `[*] Scan complete. Found N open ports.`
- זמן** — מדוד את משך הסריקה עם המודול `time`.
- (בונוס Bash)** — כתוב סקריפט Bash שמריץ קודם את ה-Ping Sweep, ואז מריץ את ה-Port Scanner על כל מכונה חיה שנמצאה.

פלט רצוי לדוגמה:

```
[*] Enter target: 127.0.0.1
[*] Ports 1-1024
[+] Port 80 is OPEN (HTTP)
[+] Port 8000 is OPEN
[*] Scan complete. Found 2 open ports in 2.3s
```

בניית שני הכלים האלה = יסוד המעבר לשלב הסריקה (מודול 6), שם נשתמש ב-nmap המקצועי. 🍰

רשימת בקרה — לפני מעבר למודול 5 ✅

- ☐ אני כותב ומריץ סקריפט Bash עם Shebang, משתנים, תנאים ולולאות
- ☐ בניתי והרצתי את ה-Ping Sweep
- ☐ אני מבין טיפוסים ומבני נתונים ב-Python

- ☐ אני כותב פונקציות ולולאות ב-Python
- ☐ בניתי והרצתי את ה-Port Scanner והבנתי את הקוד
- ☐ אני יודע מתי להשתמש ב-Bash ומתי ב-Python

פתרונות מלאים: [solutions.md](#)

פתרונות מלאים — מודול 4: סקריפטינג

נסה לבד ב- [missions.md](#) קודם.

חלק א' — Bash

4.1

```
#!/bin/bash
echo "Hello Hacker"
```

```
chmod +x hello.sh
./hello.sh
```

4.2

```
#!/bin/bash
read -p "Enter name: " name
echo "Welcome, $name"
```

4.3

```
#!/bin/bash
if [ "$1" -eq 80 ]; then
    echo "HTTP"
elif [ "$1" -eq 443 ]; then
    echo "HTTPS"
else
    echo "Unknown"
fi
```

הרצה: `443 ./script.sh` → `HTTPS`

```
#!/bin/bash
for i in $(seq 1 5); do
    echo "Pinging 10.0.0.$i"
done
```

חלק ב' — Python בסיסי

```
target = "10.0.0.5"
print(f"[*] Target is {target}")
```

```
banner = "Apache/2.4.49 (Debian)"
version = banner.split("/")[1].split(" ")[0]
print(version)    # 2.4.49
```

הסבר: `split("/")` → `['Apache', '2.4.49 (Debian)']` ; לוקחים אינדקס 1, ואז `split(" ")` [0] לוקח את החלק לפני הרווח.

```
services = {21: "FTP", 22: "SSH", 80: "HTTP", 443: "HTTPS"}
for port, name in services.items():
    print(f"Port {port} -> {name}")
```

חלק ג' — לוגיקה ולולאות

```
def check_port(port):
    services = {21: "FTP", 22: "SSH", 80: "HTTP", 443: "HTTPS"}
    return services.get(port, "Unknown")
```

`dict.get(key, default)` מחזיר את הערך אם קיים, אחרת את ברירת המחדל.

```
for port in range(20, 26):
    print(f"Port {port}: {check_port(port)}")
```

חלק ד' — הכלים האמיתיים

4.10 — הרצה:

```
chmod +x ping_sweep.sh
./ping_sweep.sh 192.168.1
```

מספר המכונות החיות תלוי במעבדה שלך — לפחות Kali וה-Gateway אמורים לענות.

4.11

```
# טרמינל 1:
python3 -m http.server 8000
# טרמינל 2:
python3 scanner.py      # הון 127.0.0.1
```

תראה Port 8000 is OPEN [+] (ואולי פורטים נוספים שרצים).

4.12 — connect_ex מחזיר 0 כשהפורט פתוח (החיבור הצליח). - settimeout(0.5) מגביל את ההמתנה לחיבור — מונע היתקעות על פורט מסונן ומאיץ את הסריקה. - try/except KeyboardInterrupt מאפשר לעצור בנקישת Ctrl+C בצורה נקייה במקום קריסה מכווערת.

```
#!/usr/bin/env python3
import socket, sys, time

services = {21:"FTP",22:"SSH",80:"HTTP",443:"HTTPS",3389:"RDP",8000:"HTTP-alt"}

target = input("[*] Enter target: ")
start_port = int(input("[*] Start port: "))
end_port = int(input("[*] End port: "))

print(f"[*] Scanning {target} ports {start_port}-{end_port}")
open_count = 0
start_time = time.time()

try:
    for port in range(start_port, end_port + 1):
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(0.5)
        if s.connect_ex((target, port)) == 0:
            name = services.get(port, "")
            print(f"[+] Port {port} is OPEN ({name})" if name else f"[+] Port {port} is OPEN")
            open_count += 1
        s.close()
except KeyboardInterrupt:
    print("\n[!] Stopped by user"); sys.exit()

elapsed = round(time.time() - start_time, 1)
print(f"[*] Scan complete. Found {open_count} open ports in {elapsed}s")
```

בונוס Bash — שרשור Ping Sweep + Scanner

```
#!/bin/bash
# מוצא מכונות חיות, ואז מריץ את סורק הפיתון על כל אחת
network=$1
for ip in $(seq 1 254); do
    if ping -c 1 -W 1 "$network.$ip" > /dev/null 2>&1; then
        echo "[+] $network.$ip is UP - scanning..."
        echo "$network.$ip" | python3 scanner.py
    fi
done
```

עברת? מצוין — יש לך עכשיו בסיס סקריפטינג אמיתי. המשך למודול 5.